

=====

EMPLOYEE PAYSIP - AUG 2026

=====

Emp ID : 1003
Name :
Department :
Grade : B
Status : Active

Basic Salary : Rs. 10001.00
Allowances : Rs. 3000.30
Gross Salary : Rs. 13001.30

Tax Deduction : Rs. 0.00
Net Salary : Rs. 13001.30

=====

is no longer active. Payroll skipped.
Total Employees : 3

=====

EMPLOYEE PAYSIP - AUG 2026

=====

Emp ID : 1001
Name : Sakshi
Department : HR
Grade : B
Status : Active

Basic Salary : Rs. 10001.00
Allowances : Rs. 3000.30
Gross Salary : Rs. 13001.30

Tax Deduction : Rs. 0.00
Net Salary : Rs. 13001.30

=====

=====

EMPLOYEE PAYSIP - AUG 2026

=====

Emp ID : 1002
Name :
Department :
Grade : B
Status : Active

Basic Salary : Rs. 10001.00
Allowances : Rs. 3000.30
Gross Salary : Rs. 13001.30

Tax Deduction : Rs. 0.00
Net Salary : Rs. 13001.30

=====

=====

EMPLOYEE PAYSIP - AUG 2026

=====

Emp ID : 1003
Name :
Department :
Grade : B
Status : Active

```

        getline(cin, tempStr);
        setName(tempStr);
        cout << "Enter department: ";
        getline(cin, tempStr);
        setDepartment(tempStr);
        cout << "Enter grade: ";
        cin >> tempChar;
        cin.ignore();
        setGrade(tempChar);
        cout << "Enter basic salary: ";
        cin >> tempDouble;
        cin.ignore();
        setBasicSalary(tempDouble);
    }
};

int Employee::employeeCount = 0;

int main() {
    Employee e1;
    Employee* e2 = new Employee();
    Employee* e3 = new Employee();

    e1.acceptDetails();
    e2->acceptDetails();
    e3->acceptDetails();
    e1.printPayslip();
    e2->printPayslip();
    e3->printPayslip();
    e3->deactivate();
    if (!e3->getIsActive())
        cout << e3->getName() << " is no longer active. Payroll skipped." << endl;
    cout << "Total Employees : " << Employee::getEmployeeCount() << endl;
    delete e2;
    delete e3;
    return 0;
}

```

```

    }
}

double computeNetSalary() const {
    return computeGrossSalary() - computeTax();
}

void printPayslip() const {
    cout << fixed << setprecision(2);
    cout << "=====\n";
    cout << "          EMPLOYEE PAYSIP - AUG 2026\n";
    cout << "=====\n";
    cout << "Emp ID      : " << empId << "\n";
    cout << "Name        : " << name << "\n";
    cout << "Department  : " << department << "\n";
    cout << "Grade       : " << grade << "\n";
    cout << "Status      : " << (isActive ? "Active" : "Inactive") << "\n";
    cout << "-----\n";
    cout << "Basic Salary : Rs. " << basicSalary << "\n";
    cout << "Allowances   : Rs. " << computeAllowances() << "\n";
    cout << "Gross Salary : Rs. " << computeGrossSalary() << "\n";
    cout << "-----\n";
    cout << "Tax Deduction : Rs. " << computeTax() << "\n";
    cout << "Net Salary    : Rs. " << computeNetSalary() << "\n";
    cout << "=====\n\n";
}

static int getEmployeeCount() {
    return employeeCount;
}

void acceptDetails() {
    string tempStr;
    char tempChar;
    double tempDouble;
    cout << "Enter name: ";
    getline(cin, tempStr);
    setName(tempStr);
    cout << "Enter department: ";
    getline(cin, tempStr);
    setDepartment(tempStr);
    cout << "Enter grade: ";
    cin >> tempChar;
    cin.ignore();
    setGrade(tempChar);
    cout << "Enter basic salary: ";
    cin >> tempDouble;
}

```

```

        cout << "ERROR: Invalid grade '" << g
              << "'. Accepted values: A, B, C, D.\n";
    }
}

void setBasicSalary(double salary) {
    if (salary > 10000 && salary < 500000) {
        basicSalary = salary;
    } else {
        cout << "ERROR: Salary must be between Rs.10,000 and Rs.5,00,000. "
              << "Value rejected.\n";
    }
}

void deactivate() {
    isActive = false;
}

int getEmpId() const { return empId; }
string getName() const { return name; }
string getDepartment() const { return department; }
char getGrade() const { return grade; }
double getBasicSalary() const { return basicSalary; }
bool getIsActive() const { return isActive; }
double computeAllowances() const {
    switch (grade) {
        case 'A': return basicSalary * 0.40;
        case 'B': return basicSalary * 0.30;
        case 'C': return basicSalary * 0.20;
        case 'D': return basicSalary * 0.10;
        default: return 0;
    }
}

double computeGrossSalary() const {
    return basicSalary + computeAllowances();
}

double computeTax() const {
    double gross = computeGrossSalary();
    if (gross <= 50000) {
        return 0;
    } else if (gross <= 100000) {
        return (gross - 50000) * 0.10;
    } else {
        return 5000 + (gross - 100000) * 0.20;
    }
}

```

```

#include <iostream>
#include <string>
#include <iomanip>
using namespace std;
class Employee {
private:
    int empId;
    string name;
    string department;
    char grade;
    double basicSalary;
    bool isActive;
    static int employeeCount;
public:
    Employee() {
        employeeCount++;
        empId = 1000 + employeeCount;
        name = "";
        department = "";
        grade = 'D';
        basicSalary = 10001;
        isActive = true;
    }
    void setName(const string& n) {
        if (n.empty()) {
            cout << "ERROR: Name cannot be empty. Value rejected.\n";
            return;
        }
        name = n;
    }
    void setDepartment(const string& dept) {
        if (dept == "Engineering" || dept == "HR" ||
            dept == "Finance" || dept == "Operations") {
            department = dept;
        } else {
            cout << "ERROR: '" << dept << "' is not a registered department.\n";
        }
    }
    void setGrade(char g) {
        if (g == 'A' || g == 'B' || g == 'C' || g == 'D') {
            grade = g;
        } else {
            cout << "ERROR: Grade must be A, B, C, or D.\n";
        }
    }
};

```

<terminated> (exit value: 0) Day1.1.exe [C/C++ Appli

Waypoint 1 (20, 10)

Distance from home : 22.3607

Inside safe zone : YES

Waypoint 2 (60, 30)

Distance from home : 67.082

Inside safe zone : NO

Waypoint 3 (-45, 40)

Distance from home : 60.208

Inside safe zone : NO

```

#include<iostream>
#include<cmath>
using namespace std;
inline double distanceBetween(double x1, double y1, double x2, double y2){
    return sqrt( pow(x2-x1,2) + pow(y2-y1,2) );
}
inline double toRadians(double degrees){
    return degrees * (M_PI / 180.0);
}
inline double clamp(double value, double minVal, double maxVal){
    if(value < minVal) return minVal;
    if(value > maxVal) return maxVal;
    return value;
}
inline bool isInSafeZone(double x, double y, double cx, double cy, double radius){
    double dist = distanceBetween(x,y,cx,cy);
    return dist <= radius;
}
int main(){
    double homeX=0.0, homeY=0.0;
    double safeRadius=50.0;
    double wx[3] = {20.0, 60.0, -45.0};
    double wy[3] = {10.0, 30.0, 40.0};

    for(int i=0;i<3;i++){
        double dist = distanceBetween(homeX, homeY, wx[i], wy[i]);
        bool safe = isInSafeZone(wx[i], wy[i], homeX, homeY, safeRadius);
        cout<<"Waypoint "<<i+1<<" ("<<wx[i]<<","<<wy[i]<<")"<<endl;
        cout<<" Distance from home : "<<dist<<endl;
        cout<<" Inside safe zone : "<<(safe ? "YES" : "NO")<<endl;
    }
    return 0;
}

```


<terminated> (exit value: 0) Day1.1.exe [C/C++ Application]

Calibration Min : 8

Calibration Max : 67

```

7 #include<iostream>
8 using namespace std;
9 bool parsePacket(const int* rawData, int size, int** outMin, int** outMax){
10     if(size<=0){
11         return false;
12     }
13     const int* minPtr = rawData;
14     const int* maxPtr = rawData;
15     for(int i=1;i<size;i++){
16         if(rawData[i] < *minPtr){
17             minPtr = &rawData[i];
18         }
19         if(rawData[i] > *maxPtr){
20             maxPtr = &rawData[i];
21         }
22     }
23     *outMin = (int*)minPtr;
24     *outMax = (int*)maxPtr;
25     return true;
26 }
27 int main(){
28     int packet[] = {45, 12, 67, 8, 55, 31};
29     int* minPtr = nullptr;
30     int* maxPtr = nullptr;
31     if(parsePacket(packet, 6, &minPtr, &maxPtr)){
32         cout<<"Calibration Min : "<<*minPtr<<endl;
33         cout<<"Calibration Max : "<<*maxPtr<<endl;
34     }
35     return 0;
36 }

```

```
regPtr1 (statusReg) value: 177  
regPtr1 repointed to dataReg: 202  
regPtr2 (controlReg) new value: 255  
regPtr3 (statusReg) value: 177
```

```
#include<iostream>
using namespace std;
int main(){
    int statusReg = 0b10110001;
    int controlReg = 0b00000000;
    int dataReg = 0b11001010;
    const int* regPtr1 = &statusReg;
    cout<<"regPtr1 (statusReg) value: "<<*regPtr1<<endl;
    regPtr1 = &dataReg;
    cout<<"regPtr1 repointed to dataReg: "<<*regPtr1<<endl;
    int* const regPtr2 = &controlReg;
    *regPtr2 = 0b11111111;
    cout<<"regPtr2 (controlReg) new value: "<<*regPtr2<<endl;
    const int* const regPtr3 = &statusReg;
    cout<<"regPtr3 (statusReg) value: "<<*regPtr3<<endl;
    return 0;
}
```

```
Before normalise: 0.5 -1.2 0.8 -0.3 1 -0.9 0.1  
RMS value: 0.778276  
Zero crossings: 6  
After normalise: 0.416667 -1 0.666667 -0.25 0.833333 -0.75 0.0833333  
After applyGain(2.0): 0.833333 -2 1.33333 -0.5 1.66667 -1.5 0.166667
```

```
    cout<<endl;
}

int main(){
    double testSignal[] = {0.5, -1.2, 0.8, -0.3, 1.0, -0.9, 0.1};
    int n = 7;
    cout<<"Before normalise: ";
    printArray(testSignal, n);
    double rms = computeRMS(testSignal, n);
    cout<<"RMS value: "<<rms<<endl;
    int crossings = countZeroCrossings(testSignal, n);
    cout<<"Zero crossings: "<<crossings<<endl;
    normalise(testSignal, n);
    cout<<"After normalise: ";
    printArray(testSignal, n);
    applyGain(testSignal, n, 2.0);
    cout<<"After applyGain(2.0): ";
    printArray(testSignal, n);
    return 0;
}
```

```

#include<iostream>
#include<cmath>
using namespace std;
double computeRMS(double* signal, int n){
    double sum=0;
    for(double* ptr=signal; ptr<signal+n; ptr++){
        sum += (*ptr) * (*ptr);
    }
    return sqrt(sum/n);
}

void normalise(double* signal, int n){
    double maxAbs=0;
    for(double* ptr=signal; ptr<signal+n; ptr++){
        if(fabs(*ptr) > maxAbs){
            maxAbs = fabs(*ptr);
        }
    }
    for(double* ptr=signal; ptr<signal+n; ptr++){
        *ptr = *ptr / maxAbs;
    }
}

int countZeroCrossings(double* signal, int n){
    int count=0;
    for(int i=0;i<n-1;i++){
        if( *(signal+i) * *(signal+i+1) < 0 ){
            count++;
        }
    }
    return count;
}

void applyGain(double* signal, int n, double gainFactor){
    for(double* ptr=signal; ptr<signal+n; ptr++){
        *ptr = *ptr * gainFactor;
    }
}

void printArray(double* signal, int n){
    for(int i=0;i<n;i++){
        cout<< *(signal+i) <<" ";
    }
    cout<<endl;
}

int main(){
    double testSignal[] = {0.5, -1.2, 0.8, -0.3, 1.0, -0.9, 0.1};
    int n = 7;
    cout<<"Before normalise: ";
    printArray(testSignal, n);
    double rms = computeRMS(testSignal, n);

```

<terminated> (exit value: 0) Day1.1.exe [C/C++ Application] D:\C++\

--- V1: Call by Value ---

Before : A=55 B=12

After : A=55 B=12 <- values unchanged

--- V2: Call by Reference ---

Before : A=55 B=12

After : A=12 B=55 <- values swapped

--- V3: Call by Pointer ---

Before : A=12 B=55

After : A=55 B=12 <- values swapped back


```

#include<iostream>
using namespace std;
void resetSensorPairV1(int reading1, int reading2){
    int temp = reading1;
    reading1 = reading2;
    reading2 = temp;
}
void resetSensorPairV2(int& reading1, int& reading2){
    int temp = reading1;
    reading1 = reading2;
    reading2 = temp;
}
void resetSensorPairV3(int* reading1, int* reading2){
    int temp = *reading1;
    *reading1 = *reading2;
    *reading2 = temp;
}
int main(){
    int A=55, B=12;
    cout<<"--- V1: Call by Value ---"<<endl;
    cout<<"Before : A="<<A<<" B="<<B<<endl;
    resetSensorPairV1(A,B);
    cout<<"After  : A="<<A<<" B="<<B<<"    <- values unchanged"<<endl;
    cout<<"\n--- V2: Call by Reference ---"<<endl;
    cout<<"Before : A="<<A<<" B="<<B<<endl;
    resetSensorPairV2(A,B);
    cout<<"After  : A="<<A<<" B="<<B<<"    <- values swapped"<<endl;
    cout<<"\n--- V3: Call by Pointer ---"<<endl;
    cout<<"Before : A="<<A<<" B="<<B<<endl;
    resetSensorPairV3(&A,&B);
    cout<<"After  : A="<<A<<" B="<<B<<"    <- values swapped back"<<endl;
    return 0;
}

```

<terminated> (exit value: 0) Day1.1.exe [C/C++ Application] D:\C++\Day1.1\Debug\Day1.1.exe (29/08/26, 10:04 am)

Config : Warn=30C Critical=45C Readings=10

Results : Normal:2 Warning:5 Critical:2 Shutdown:1

<terminated> (exit value: 1) Day1.1.exe [C/C++ Application] D:\C++\Day1.1\Debug\Day1.1.exe (29/08/26, 10:00)

Usage : ./sensor_monitor <warn_threshold> <critical_threshold> <num_readings>

Error : Missing arguments.

```

#include<iostream>
#include<cstdlib>
using namespace std;
int main(int argc, char* argv[]){

    if(argc<4){
        cout<<"Usage : ./sensor_monitor <warn_threshold> <critical_threshold> <num_readings>"<<endl;
        cout<<"Error : Missing arguments."<<endl;
        return 1;
    }

    int warnT=atoi(argv[1]);
    int criticalT=atoi(argv[2]);
    int numReadings=atoi(argv[3]);

    if(warnT>=criticalT){
        cout<<"Error : warn_threshold must be less than critical_threshold."<<endl;
        return 1;
    }
    if(numReadings<1 || numReadings>500){
        cout<<"Error : num_readings must be between 1 and 500."<<endl;
        return 1;
    }

    cout<<"Config : Warn="<<warnT<<"C Critical="<<criticalT<<"C Readings="<<numReadings<<endl;

    int normal=0, warning=0, critical=0, shutdown=0;
    for(int i=0;i<numReadings;i++){
        int t=rand()%70;

        if(t<warnT){
            normal++;
        }else if(t<criticalT){
            warning++;
        }else if(t<60){
            critical++;
        }else{
            shutdown++;
        }
    }

    cout<<"Results : Normal:"<<normal<<" Warning:"<<warning<<" Critical:"<<critical<<" Shutdown:"<<shutdown<<endl;

    return 0;
}

```

day6_notes.md X day5_notes.md X

Day5 > day5_notes.md > abc # 28/8/26 DAY 5: > abc ## Default Constructor

```
1  # 28/8/26 DAY 5:
134 ## Constructor Concept
140
141 ## Default Constructor
142 A default constructor is a constructor that can be called with no arguments. It is either explicitly defined
    by the programmer or automatically provided by the compiler if no other constructors are defined.
143
144 ```cpp
145 class MyClass {
146 public:
147     MyClass() { // Default constructor
148         // Initialize data members
149     }
150 };
151 ```
152 ## Parameterized Constructor
153 A parameterized constructor is a constructor that takes parameters to initialize an object with specific
    values. It allows you to create objects with different initial states.
154
155 ```cpp
156 class MyClass {
157 public:
158     MyClass(int a, int b) { // Parameterized constructor
159         // Initialize data members with the provided values
160     }
161 };
162 ```
163 ## Multiple Constructors
164 A class can have multiple constructors, each with a different set of parameters. This allows for creating
    objects in different ways. The concept of having multiple constructors is known as constructor overloading.
```

```

7 #include <iostream>
8 #include <cstdlib> // for atoi, rand
9 using namespace std;
0 int main(int argc, char* argv[]) {
1     // Step 1: check arguments are present
2     if (argc < 4) {
3         cout << "Usage : ./sensor_monitor <warn_threshold> <critical_threshold> <num_readings>" << endl;
4         cout << "Error : Missing arguments." << endl;
5         return 1;
6     }
7     int warn = atoi(argv[1]);
8     int critical = atoi(argv[2]);
9     int numReadings = atoi(argv[3]);
0     if (warn >= critical) {
1         cout << "Error : warn_threshold must be less than critical_threshold." << endl;
2         return 1;
3     }
4     if (numReadings < 1 || numReadings > 500) {
5         cout << "Error : num_readings must be between 1 and 500." << endl;
6         return 1;
7     }
8     cout << "Config : Warn=" << warn << "C Critical=" << critical << "C Readings=" << numReadings << endl;
9     int normal = 0, warning = 0, criticalCount = 0, shutdown = 0;
0     for (int i = 0; i < numReadings; i++) {
1         int temp = rand() % 70;
2         if (temp < warn) normal++;
3         else if (temp < critical) warning++;
4         else if (temp < 60) criticalCount++;
5         else shutdown++;
6     }
7     cout << "Results : Normal:" << normal << " Warning:" << warning
8         << " Critical:" << criticalCount << " Shutdown:" << shutdown << endl;
9     return 0;
0 }
1

```

```

double bestAvg=-1;
int bestFloor=0;
for(int floor=0;floor<3;floor++){
    double sum=0;
    for(int room=0;room<3;room++){
        sum=sum+grid[floor][room];
    }
    double avg=sum/3;
    if(avg>bestAvg){
        bestAvg=avg;
        bestFloor=floor;
    }
}
cout<<"Hottest Floor : Floor "<<bestFloor+1<<" (avg "<<bestAvg<<"C)"<<endl;

int warningCount=0;
for(int floor=0;floor<3;floor++){
    for(int room=0;room<3;room++){
        if(grid[floor][room]>=30){
            warningCount++;
        }
    }
}
cout<<"Rooms at WARNING or above : "<<warningCount<<endl;

return 0;
}

```

```

#include<iostream>
using namespace std;
int main(){
    double grid[3][3];

    for(int floor=0;floor<3;floor++){
        for(int room=0;room<3;room++){
            cout<<"Floor "<<floor+1<<" Room "<<room+1<<": ";
            cin>>grid[floor][room];
        }
    }
    cout<<"\n          Room1    Room2    Room3"<<endl;
    for(int floor=0;floor<3;floor++){
        cout<<"Floor "<<floor+1<<" : ";
        for(int room=0;room<3;room++){
            cout<<grid[floor][room]<<"    ";
        }
        cout<<endl;
    }

    double hottest=grid[0][0];
    int hotFloor=0, hotRoom=0;
    for(int floor=0;floor<3;floor++){
        for(int room=0;room<3;room++){
            if(grid[floor][room]>hottest){
                hottest=grid[floor][room];
                hotFloor=floor;
                hotRoom=room;
            }
        }
    }
    cout<<"\nHottest Room : Floor "<<hotFloor+1<<", Room "<<hotRoom+1<<" -> "<<hottest<<"C"<<endl;

    double bestAvg=-1;
    int bestFloor=0;
    for(int floor=0;floor<3;floor++){
        double sum=0;
        for(int room=0;room<3;room++){
            sum=sum+grid[floor][room];
        }
        double avg=sum/3;
        if(avg>bestAvg){
            bestAvg=avg;
            bestFloor=floor;
        }
    }
    cout<<"Hottest Floor : Floor "<<bestFloor+1<<" (avg "<<bestAvg<<"C)"<<endl;

```


<terminated> (exit value: 0) Day1.1.exe [C/C++ Application] D:\C+

Floor 1 Room 1: 24.0

Floor 1 Room 2: 31.5

Floor 1 Room 3: 28.0

Floor 2 Room 1: 45.0

Floor 2 Room 2: 22.0

Floor 2 Room 3: 30.0

Floor 3 Room 1: 19.0

Floor 3 Room 2: 27.5

Floor 3 Room 3: 50.2

	Room1	Room2	Room3
Floor 1 :	24	31.5	28
Floor 2 :	45	22	30
Floor 3 :	19	27.5	50.2

Hottest Room : Floor 3, Room 3 -> 50.2C

Hottest Floor : Floor 2 (avg 32.3333C)

Rooms at WARNING or above : 4

```

7 #include<iostream>
8 using namespace std;
9 void hottestroom(double arr[3][3]){
10     int max = arr[0][0];
11     int floor = 0;
12     int room = 0;
13     for(int i = 0; i < 3; i++){
14         for(int j = 0; j < 3; j++){
15             if(max < arr[i][j]){
16                 max = arr[i][j];
17                 floor = i+1;
18                 room = j+1;
19             }
20         }
21     }
22     cout << " Hottest Room: " << " Floor " << floor << " , " << "Room" << room << "->" << max << "C" << endl;
23 }
24 void hottestfloor(double arr[3][3]){
25     double max = (arr[0][0] + arr[0][1] + arr[0][2]) / 3;
26     int floor = 1;
27     for(int i = 1; i < 3; i++){
28         int sum = 0;
29         for(int j = 0; j < 3; j++){
30             sum += arr[i][j];
31         }
32         double avg = sum / 3;
33         if(avg > max){
34             max = avg;
35             floor = i+1;
36         }
37     }
38     cout << "hottest floor: floor" << floor << "(avg " << max << "C)" << endl;
39 }
40 int main(){
41     cout << " enter data of 9 rooms " << endl;
42     double arr[3][3];
43     for(int i = 0; i < 3; i++){
44         for(int j = 0; j < 3; j++){
45             cin >> arr[i][j];
46         }
47     }
48     cout << "\tRoom1 \tRoom2 \tRoom3" << endl;
49     for(int i = 0; i < 3; i++){
50         for(int j = 0; j < 3; j++){
51             cout << " Floor 1: ";
52             cout << arr[i][j] << "\t";
53         }
54         cout << endl;

```

```

7 #include<iostream>
8 using namespace std;
9 int main(){
10     int n;
11     cout<<"Number of readings: ";
12     cin>>n;
13     double temp[100];
14     for(int i=0;i<n;i++){
15         cout<<"Enter readings "<<i<<" : ";
16         cin>>temp[i];
17     }
18     cout<<"\nReadings entered : "<<n<<endl;
19     int skipped=0;
20     cout<<"Valid readings : ";
21     for(int i=0;i<n;i++){
22         if(temp[i]<0){
23             skipped++;
24             continue;
25         }
26         cout<<temp[i]<<" ";
27     }
28     cout<<endl;
29     cout<<"Skipped (errors) : "<<skipped<<endl;
30
31     for(int i=0;i<n;i++){
32         if(temp[i]>=45){
33             cout<<"\nFirst CRITICAL : Index "<<i<<" -> "<<temp[i]<<"C"<<endl;
34             break;
35         }
36     }
37
38     double miniVal=1, maxVal=-1, sum=0;
39     int count=0;
40     for(int i=0;i<n;i++){
41         if(temp[i]<0){
42             continue;
43         }
44         if(temp[i]<miniVal){
45             miniVal=temp[i];
46         }
47         if(temp[i]>maxVal){
48             maxVal=temp[i];
49         }
50         sum=sum+temp[i];
51         count++;
52     }
53     double avg=sum/count;

```

```
terminated (exit value 0) by (pid) 11000 [0/0] Application: 0100 [0/0] 11000 [0/0] 11000 [0/0] 11000 [0/0]
Number of readings: 5
Enter readings 0: 85 25 57 62 37
Enter readings 1: Enter readings 2: Enter readings 3: Enter readings 4:
Readings entered : 5
Valid readings   : 85 25 57 62 37
Skipped (errors) : 0

First CRITICAL   : Index 0 -> 85C

Min : 1C   Max : 85C   Avg : 53.2C

Normal:0  Warning:2  Critical:1  Shutdown:2
```

<terminated> (exit value: 0) Day1.1.exe [C/C++ Appli

Enter the Range of temperature: 20

Temperature : 20 / 68

Status : NORMAL

Action : No action required

Reading : Below Average

```

switch(statusCode){
case -1:
    cout<<"Status : SENSOR_ERROR"<<endl;
    cout<<"Action : Sensor fault – check wiring"<<endl;
    break;

case 0:
    cout<<"Status : NORMAL"<<endl;
    cout<<"Action : No action required"<<endl;
    break;

case 1:
    cout<<"Status : WARNING"<<endl;
    cout<<"Action : Alert sent to supervisor"<<endl;
    break;

case 2:
    cout<<"Status : CRITICAL"<<endl;
    cout<<"Action : Cooling system triggered"<<endl;
    break;

default:
    cout<<"Status : SHUTDOWN"<<endl;
    cout<<"Action : Emergency shutdown initiated"<<endl;
}

cout<<"Reading : "<< (range>=25?"Above Average" : "Below Average")<<endl;

return 0;
}

```

```

1 #include<iostream>
2 using namespace std;
3
4 int main(){
5     double range;
6     cout<<"Enter the Range of temperature: ";
7     cin>>range;
8     double F=(range*9/5)+32;
9     cout<<"Temperature : "<<range<<" / "<<F<<endl;
10
11     int statusCode;
12
13     if(range<0){
14         statusCode= -1;
15     }else if(range>=0 && range<=29){
16         statusCode= 0;
17     }else if(range>=30 && range<=44){
18         statusCode= 1;
19     }else if(range>=45 && range<=59){
20         statusCode= 2;
21     }else{
22         statusCode= 3;
23     }
24
25     switch(statusCode){
26     case -1:
27         cout<<"Status : SENSOR_ERROR"<<endl;
28         cout<<"Action : Sensor fault – check wiring"<<endl;
29         break;
30
31     case 0:
32         cout<<"Status : NORMAL"<<endl;
33         cout<<"Action : No action required"<<endl;
34         break;
35
36     case 1:
37         cout<<"Status : WARNING"<<endl;
38         cout<<"Action : Alert sent to supervisor"<<endl;
39         break;
40
41     case 2:
42         cout<<"Status : CRITICAL"<<endl;
43         cout<<"Action : Cooling system triggered"<<endl;

```